

■ The Core Idea

Every Claude task has an **optimal model**. Choosing the wrong one wastes tokens, slows you down, and costs more — without improving results. Model selection comes down to three variables: **Capability** (what the model can do), **Speed** (how fast it responds), and **Cost / Rate Limit** (how much it consumes).

■ *Key insight from the docs: Tuning the **effort parameter** within a model is often a better optimization lever than switching models entirely — especially with newer Sonnet and Opus versions.*

■ Three Key Criteria Before Choosing

■ Capability

What features does the task actually need? (Code gen, vision, long context, sustained reasoning, computer use?)

■ Speed

How fast must responses arrive? Real-time = Haiku; interactive pair-programming = Sonnet; deep overnight work = Opus.

■ Cost / Rate Limit

Haiku is lightest on rate limits. Sonnet is moderate. Opus is heaviest. Using Opus for Haiku-level tasks burns your limit needlessly.

■ Effort Level

New lever: adjust effort (low → standard → high → xhigh → max) within a model before switching models. Opus 4.8 defaults to 'high'; use 'xhigh' for coding/agent tasks.

■ Meet the Three Models

■ Haiku

4.5 — Current

Fast, lightweight — rivals old Sonnet 4.0 reasoning at a fraction of the cost.

Rate: Lightest

- Simple, direct questions needing short answers
- Quick lookups, categorizations, data extraction
- Basic summarization and synthesis of short text
- High-volume, cost-sensitive applications
- Real-time applications with tight latency requirements
- Sub-agent tasks in larger multi-step workflows

■ Sonnet

4.6 — Current

The daily driver. Strong reasoning for everyday professional work.

Rate: Moderate

- Writing, content creation, and editing
- Coding — debugging, writing new code, refactoring
- Analysis requiring reasoning (not extreme complexity)
- Multi-step workflows and agentic tool use
- Vision tasks, document & spreadsheet creation
- Code generation and data analysis at scale
- Best default: if unsure which model to use, start here

■ Opus

4.8 — Most Capable

Large reasoning specialist. For tasks that genuinely need deep, sustained thinking.

Rate: Heaviest

- Deep research requiring multi-step reasoning across long documents
- Scientific or mathematical applications
- Complex systems engineering and large-scale code refactoring
- High-autonomy agentic work (multi-hour autonomous coding agents)
- Tasks where accuracy outweighs cost considerations
- Problems where Sonnet has already been tested and struggled
- Fast Mode available: up to 2.5× higher output speed at premium pricing

■ Understanding Rate Limits

Rate limits cap total tokens in a time window. Each model consumes at a different rate:

Model	Token Consumption	Extended Thinking	Efficiency
Haiku 4.5	Lightest	Standard	■ Most efficient
Sonnet 4.6	Moderate	Adaptive (auto-calibrated)	■■ Balanced
Opus 4.8	Heaviest	Adaptive (auto-calibrated)	■ Use selectively

■ **Extended Thinking tip:** Both Sonnet 4.6 and Opus 4.8 feature adaptive extended thinking — Claude auto-calibrates reasoning depth. Simple questions get fast answers; complex ones get more thinking. This makes leaving extended thinking on more token-efficient than before.

■ When to Use Each Model

Use Case	Best Model	Why
Debugging code	■ Sonnet	Exceptional coding capabilities + fast feedback loop
Summarizing short articles	■ Haiku	Straightforward extraction, no complex reasoning needed
Analyzing research papers	■ Opus	Deep analysis, methodology critique, forward-looking insights
Writing new feature code	■ Sonnet	Strong code generation + interactive collaboration speed
Quick syntax lookup	■ Haiku	Instant, no reasoning depth required

Use Case	Best Model	Why
Multi-hour autonomous agent	■ Opus 4.8	Long-horizon planning, high-autonomy agentic work
Data analysis + visualization	■ Sonnet	Frontier intelligence at scale for enterprise workflows
Real-time chatbot / support	■ Haiku	Speed priority, high-volume, cost-sensitive

■ ■ Two Selection Strategies (from the API Docs)

1

Start Fast & Scale Up

1. Begin with Claude Haiku 4.5
2. Test your use case thoroughly
3. Evaluate if performance meets requirements
4. Upgrade only if specific capability gaps appear

Best for:

- Initial prototyping and development
- Tight latency requirements
- Cost-sensitive implementations
- High-volume, straightforward tasks

2

Start Capable & Optimize Down

1. Implement with Claude Opus 4.8
2. Optimize prompts for maximum quality
3. Evaluate if performance meets requirements
4. Lower effort or downgrade models once workflow is optimized

Best for:

- Complex reasoning and scientific tasks
- Tasks where accuracy outweighs cost
- Advanced coding and high-autonomy agentic work
- Problems with nuanced understanding requirements

■ ■ The Effort Parameter — A Newer Lever

Recent Opus and Sonnet models support an **effort parameter** that trades intelligence for latency and cost *within the same model*. The API docs note this is often a better optimization lever than switching models entirely.

Level	Speed	Cost	Best Use
low	Fastest	Lowest	Simple lookups, quick tasks
standard	Fast	Low	Default for most daily tasks
high	Moderate	Medium	Opus 4.8 default; complex analysis
xhigh	Slower	High	Opus 4.8: coding & agentic work ■
max	Slowest	Highest	Maximum intelligence, critical decisions

■ **Pro tip (from API docs):** On Claude Opus 4.8, use **xhigh** (between 'high' and 'max') for coding, agentic use cases, and intelligence-demanding tasks — it's the sweet spot.

■ Model Access by Plan

Plan	Models Available	Rate Limit
Free	Haiku + Sonnet	Lower
Pro / Max	Haiku + Sonnet + Opus	Higher
API / Platform	All models via API strings	By tier

■ How to Test & Decide When to Upgrade

- 1 Create Benchmarks**
Build test cases specific to your actual use case — having a good eval set is the most critical step.
- 2 Test With Real Data**
Run your actual prompts and real data, not toy examples.
- 3 Compare Across Models**
Measure accuracy, response quality, and edge case handling.
- 4 Weigh Trade-offs**
Balance performance gains against cost and speed requirements.
- 5 Re-test on New Releases**
Each Claude version is a separate training run — a task that needed Opus may now suit Sonnet, or vice versa.

■ Key Takeaways to Remember

- **Haiku** = speed + efficiency. Default for high-volume, simple, real-time tasks.
- **Sonnet** = your daily driver. Handles 80%+ of professional coding and writing work.
- **Opus** = deep reasoning specialist. Reserve for complex, accuracy-critical problems.
- ■ **Try effort tuning first** before switching models — it's often the better optimization.
- **Model balance shifts with each release** — re-test your assumptions after new versions drop.
- **Rate limits** are per-token — using the wrong model wastes your quota with zero benefit.
- **Always benchmark with real data** — toy tests don't reveal real-world performance gaps.

■ Quick Cheat Sheet

Situation	→ Use
Not sure which to pick	■ Sonnet (default)
Simple Q&A; / lookup	■ Haiku
Code debugging / writing	■ Sonnet
Complex research paper	■ Opus
Autonomous long-running agent	■ Opus 4.8 xhigh
High-volume chatbot	■ Haiku
Vision / spreadsheet tasks	■ Sonnet
Production code — accuracy critical	■ Opus or Sonnet High

